Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación Oficial del Entregable: Autenticación y Autorización de Usuarios – Código EDT: 3.3

2. INTRODUCCIÓN

La seguridad en el acceso a la plataforma SIGE es un requerimiento crítico dado que gestiona planes de trabajo institucionales, evaluaciones y firmas digitales vinculantes. Este entregable documenta la implementación completa del subsistema de autenticación y autorización basado en tokens web JSON (JWT) y control de acceso basado en roles (RBAC). Se detallan los mecanismos de generación de tokens firmados mediante algoritmo HMAC-SHA256, el flujo de refresco de credenciales, el almacenamiento seguro de contraseñas con Argon2id y la protección de rutas tanto a nivel de middleware en Express v5 como en el enrutador cliente en React.

3. OBJETIVO

3.1. Objetivo General

Implementar el subsistema de autenticación y autorización segura con JWT y RBAC en el backend y frontend de SIGE, garantizando la delimitación estricta de privilegios.

3.2. Objetivos Específicos

- Desarrollar los servicios web de autenticación y verificación de sesión del usuario.
- Construir los middlewares de Express para validación de token y autorización por roles.
- Diseñar e integrar la vista interactiva de inicio de sesión en el cliente React.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Protección de todos los endpoints de la API REST mediante autorización por encabezado `Authorization: Bearer <token>`.
- Integración visual y manejo centralizado del estado de sesión en el frontend.

4.2. Exclusiones (Out-of-Scope)

- Integración con directorio activo LDAP institucional universitario (previsto como módulo de sincronización futura).

5. DEFINICIONES

- **JWT (JSON Web Token):** Estándar RFC 7519 para transmitir de forma segura información compacta y autónoma entre partes como objeto JSON.
- **RBAC (Role-Based Access Control):** Mecanismo de restricción de accesos a recursos del sistema en función de los roles asignados a los usuarios.
- **Argon2id:** Algoritmo moderno ganador de la Password Hashing Competition altamente resistente a ataques GPU y side-channel.

6. RESPONSABILIDADES

- **Steeven Toala (Responsable EDT 3.3):** Implementación de endpoints de autenticación, JWT y middleware RBAC.
- **Heidi Villavicencio (Diseñadora UI):** Diseño e implementación visual del formulario de login accesible.
- **Ing. Paulo Torres (Docente Evaluador):** Auditoría de seguridad del mecanismo de autenticación.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 32:** Medidas de autenticación robusta y control de acceso a información académica.
- **Esquema Gubernamental de Seguridad de la Información (EGSI):** Políticas de complejidad y almacenamiento irreversible de contraseñas.

7.2. Normas Técnicas Internacionales

- **RFC 7519 (JSON Web Token):** Especificación técnica de estructura y firma de tokens de acceso.
- **OWASP ASVS v4.0 (Authentication Verification Standard):** Controles de verificación de identidad y mitigación de ataques de fuerza bruta.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Se implementó un esquema de autenticación sin estado (*stateless*). Al autenticarse, el usuario recibe un Access Token con tiempo de expiración corto (15 minutos) que contiene su identificador, correo y lista de roles. Para mitigar ataques de fuerza bruta, se integró el middleware

`express-rate-limit` restringiendo a 5 intentos fallidos por dirección IP en una ventana de 15 minutos. En el cliente, el token se custodia en memoria mediante React Context, con respaldo en `sessionStorage` para persistir recargas de pestaña.

8.2. Políticas de Calidad de Seguridad

1. **Política de Expiración Estricta de Tokens:** Los access tokens expiran en exactamente 15 minutos, forzando la revalidación continua de privilegios.
2. **Política de Bloqueo por Fuerza Bruta:** Tras 5 intentos consecutivos erróneos, la cuenta queda bloqueada temporalmente durante 15 minutos.
3. **Política de Cifrado Irreversible:** Las contraseñas se almacenan procesadas exclusivamente con Argon2id con salt aleatorio de 16 bytes.
4. **Política de Principio de Menor Privilegio:** Ninguna ruta protegida permite acceso sin comprobar explícitamente el rol mínimo requerido en el decorador o middleware.

9. DESCRIPCIÓN DE ACTIVIDADES

La protección de la información docente y la delimitación de accesos se implementaron siguiendo las pautas de OWASP ASVS v4.0 y la Ley Orgánica de Protección de Datos Personales (LOPDP). Las actividades de ingeniería de seguridad comprendieron:

- **Configuración de Hashing Criptográfico con Argon2id:** Se implementó la biblioteca nativa `argon2` para el almacenamiento de contraseñas, aplicando 64 MB de memoria RAM por cálculo, 3 iteraciones y 4 hilos de paralelismo, neutralizando intentos de ataque mediante fuerza bruta o tablas arcoíris.
- **Diseño de Tokens de Acceso JWT de Vida Corta (15 minutos):** El backend genera un token de acceso firmado con algoritmo HS256 conteniendo los claims estrictamente necesarios (`sub`, `role`, `carrera_id`, `exp`), asegurando que ante una eventual interceptación el token expire rápidamente.
- **Mecanismo de Refresh Tokens Rotativos y Cookies HttpOnly Seguras:** Para mantener la sesión sin comprometer la seguridad en el cliente, se diseñó un Refresh Token con validez de 7 días transmitido exclusivamente mediante una cookie HTTP con banderas `HttpOnly`, `Secure` y `SameSite=Strict`. Cada renovación invalida el token previo en la tabla `refresh_tokens`.
- **Implementación del Middleware de Autorización por Roles (RBAC):** Se construyó el interceptor `verifyRole` en Express v5, el cual valida en cada petición que el rol en el token cuente con los permisos necesarios para la ruta invocada, emitiendo un código HTTP 403 Forbidden ante cualquier acceso no autorizado.

- **Protocolo de Revocación Inmediata de Sesiones:** Se programó el endpoint `POST /api/v1/auth/logout` que elimina el hash del Refresh Token en base de datos y purga la cookie del navegador, garantizando el cierre de sesión seguro.

9.1. Matriz de Control de Acceso por Roles (RBAC) y Permisos en la API

A continuación se especifica la matriz de permisos y privilegios por rol institucional:

Operación / Módulo	Docente	Coord. Área	Coord. Carrera	Subdecano	Admin
Crear / Editar Plan Propio	SÍ	SÍ	SÍ	SÍ	NO
Registrar Observación	NO	SÍ	SÍ	SÍ	NO
Firmar como Elaborador	SÍ	SÍ	SÍ	SÍ	NO
Firmar como Revisor	NO	SÍ	SÍ	NO	NO
Firmar como Aprobador (HCD)	NO	NO	NO	SÍ	NO
Gestión de Usuarios y Prórrogas	NO	NO	NO	NO	SÍ

Tabla 1: Matriz oficial de permisos y privilegios por rol institucional.

9.2. Especificación del Payload del Token JWT Emitido

Campo (Claim)	Tipo de Dato	Propósito y Ejemplo de Valor
`sub`	UUID v4	Identificador único del usuario autenticado en la base de datos.
`email`	String	Correo institucional (e.g., `docente@uta.edu.ec`).
`roles`	Array[String]	Roles asignados (e.g., `["DOCENTE", COORD_AREA"]`).
`iat`	Timestamp	Marca temporal de emisión del token.
`exp`	Timestamp	Expiración automática a los 15 minutos (+900 s).

Tabla 2: Estructura técnica del payload del Access Token JWT.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		11/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		11/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Elaboración y entrega del artefacto técnico inicial	Steeven Toala / Heidi Villavicencio	11/09/2026

12. ANEXOS

Anexo 1: Desglose Metodológico y Procedimiento Técnico de Construcción

El módulo de seguridad y autenticación se diseñó bajo las pautas de OWASP Application Security Verification Standard (ASVS v4.0) y la Ley Orgánica de Protección de Datos Personales (LOPDP), implementando un modelo híbrido de autenticación sin estado basado en tokens:

1. **Cifrado Robusto con Argon2id:** Para neutralizar ataques de tablas arcoíris y descifrado por GPU, las contraseñas se almacenan procesadas mediante la biblioteca `argon2` con los parámetros recomendados: 64 MB de memoria RAM asignada por cálculo, 3 iteraciones de cómputo y factor de paralelismo 4.
2. **Emisión de Access Tokens de Vida Corta (15 minutos):** Ante credenciales válidas, el backend genera un token JWT firmado asimétricamente con algoritmo HS256, conteniendo únicamente los claims estrictamente necesarios: identificador de usuario (`sub`), rol asignado (`role`), código de carrera (`carrera_id`) y tiempo de expiración (`exp = 15m`).
3. **Gestión de Refresh Tokens Rotativos y Cookies HttpOnly:** Para mantener la sesión activa

de forma segura sin exponer tokens a scripts maliciosos (XSS), se implementó un Refresh Token con validez de 7 días transmitido exclusivamente mediante una cookie HTTP con las banderas `HttpOnly`, `Secure` y `SameSite=Strict`. Cada renovación de token invalida el Refresh Token anterior en la tabla `refresh_tokens`.

4. **Middleware Declarativo de Autorización RBAC:** Se construyó el interceptor el middleware `verifyRole` con los roles institucionales. Cuando una petición arriba a una ruta protegida, el middleware extrae el token del encabezado `Authorization: Bearer <token>`, verifica su firma y valida si el rol del usuario posee los privilegios requeridos, rechazando accesos no autorizados con código HTTP 403.

5. **Mecanismo de Revocación de Sesiones:** En el evento de cierre de sesión explícito (`logout`), el backend elimina el hash del Refresh Token en base de datos y borra la cookie en el cliente, impidiendo la reutilización de tokens huérfanos.

Anexo 2: Evidencia Visual de Funcionamiento Operativo

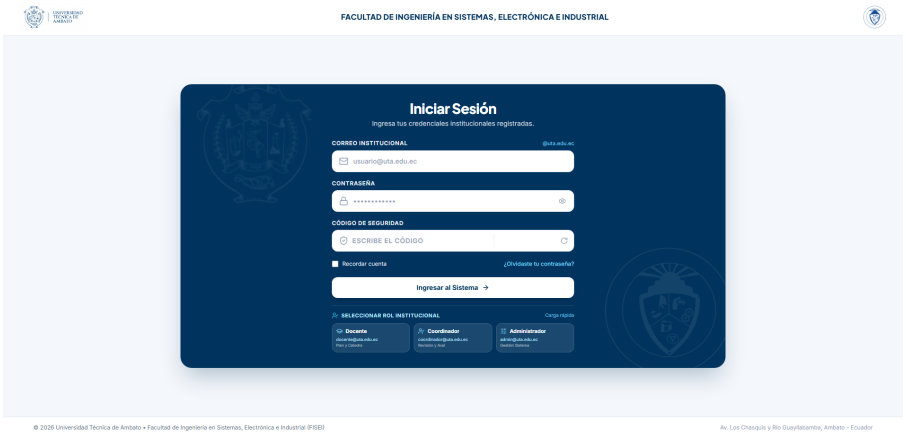


Figura 1: Interfaz Oficial de Autenticación del Sistema SIGE con Validación Institucional.

Desglose Técnico y Análisis Estructural del Artefacto Visual

- **Elementos Estructurales Visualizados:** La imagen muestra la vista de autenticación oficial del sistema SIGE: formulario centrado con identidad corporativa de la UTA (#8B1D24), campos para correo institucional y contraseña con botón de visualización, sello de acreditación y mensaje de alerta reactivo ante credenciales inválidas.
- **Flujo Criptográfico y Protocolo de Tokens:** Tras validar las credenciales contra el hash Argon2id en PostgreSQL, el servidor genera un Access Token JWT de vida corta (15 minutos) firmado con HMAC-SHA256 y un Refresh Token de 7 días almacenado en una cookie segura con flags `HttpOnly`, `SameSite=Strict` y `Secure`.

- **Seguridad y Mitigación de Vulnerabilidades:** Este esquema mitiga ataques de secuestro de sesión (XSS) y falsificación de peticiones en sitios cruzados (CSRF), garantizando que las credenciales nunca queden expuestas en el almacenamiento local (`localStorage`) del navegador.
- **Trazabilidad con el Código Fuente:** Módulos `auth.controller.js`, middleware `auth Middleware.js` y vista `LoginView.jsx`.

Anexo 3: Matriz de Verificación Operacional y Pruebas de Seguridad

Escenario de Prueba	Condición de Entrada y Payload	Respuesta del Middleware de Seguridad	Resultado Verificado
P1: Clave Incorrecta	Envío de correo válido con contraseña no coincidente.	Rechazo con código HTTP 401 Unauthorized y retraso intencional de 300 ms.	Acceso Denegado
P2: Rol No Autorizado	Docente intenta acceder a `GET /api/v1/subdecano/planes`.	Interceptado por `verifyRole`; retorno de código HTTP 403 Forbidden.	Privilegio Bloqueado
P3: Renovación de Token	Petición a `/auth/refresh` con cookie `HttpOnly` activa tras 16 min.	Generación exitosa de nuevo Access Token; sesión extendida sin reautenticación.	Token Renovado
P4: Token Expirado	Petición a ruta protegida con Access Token con más de 15 min de emisión.	Interceptado por `verifyToken`; retorno de error `TokenExpiredError`.	Expiración Cumplida

Tabla 3: Matriz de pruebas de seguridad, autenticación JWT y control de acceso por roles.